

Assignment-17.2

Task Description – 1(Student Academic Data Cleaning)

• Task:

Use AI assistance to generate a Python script that cleans and prepares a student academic dataset for analysis.

Instructions:

- Handle missing values in marks, attendance, and department.
- Remove duplicate student records based on student ID.
- Standardize text fields such as department names.
- Encode categorical variables like gender and department.

Sample Code:

```
import pandas as pd
```

```
data = pd.read_csv("students.csv")
```

```
print(data.head())
```

Expected Output:

- A cleaned and encoded Pandas DataFrame suitable for academic performance analysis.

The screenshot displays a code editor with a Python script for student data cleaning. The script includes imports for pandas, numpy, sklearn.preprocessing, and warnings. It defines functions to load the dataset, display initial information, and handle missing values. The AI assistant chat interface on the right provides guidance and feedback on the script.

```
Task1.py > ...
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import LabelEncoder
4 import warnings
5 warnings.filterwarnings('ignore')
6
7 # Load the dataset
8 def load_data(filepath):
9     """Load the student academic dataset from CSV file."""
10    try:
11        data = pd.read_csv(filepath)
12        print("Dataset loaded successfully!")
13        print(f"Initial shape: {data.shape}\n")
14        return data
15    except FileNotFoundError:
16        print(f"Error: File '{filepath}' not found.")
17        return None
18
19 # Display initial data info
20 def display_initial_info(data):
21     """Display initial dataset information."""
22     print("=" * 60)
23     print("INITIAL DATA OVERVIEW")
24     print("=" * 60)
25     print("\nFirst few records:")
26     print(data.head())
27     print("\nDataset Info:")
28     print(data.info())
29     print("\nMissing Values:")
30     print(data.isnull().sum())
31     print("\nDataset Shape:", data.shape)
32     print()
33
34 # Handle missing values
35 def handle_missing_values(data):
36     """Handle missing values in marks, attendance, and department."""
37     print("=" * 60)
38     print("HANDLING MISSING VALUES")
39     print("=" * 60)
```

PYTHON SCRIPT FOR STUDENT DATA C...

I'll help you create a comprehensive Python script for student academic data cleaning. Let me first check the current state of Task1.py.

Reviewed Task1.py

Now I'll create a comprehensive data cleaning script that addresses all the requirements:

Created Task1.py

Now let me create a sample `students.csv` file so you can test the script immediately:

Created students.csv

Now let me test the script to verify it works correctly:

Running 'python Task1.py'

Ran 'python Task1.py'

Configured Python environment

Let me configure the Python environment for your workspace:

Configured a Python environment

Creating a Virtual Environment

+ Task1.py

Describe what to build

+ 🔍 Auto 🔍

Local 🔍 Default Approvals 🔍

Ln 246, Col 1 Spaces: 4 UTF-8 CRLF {} Python 3.14.2 (venv) .venv (3.14.2) Go Live

students.csv

```
1 student_id,name,gender,department,marks,attendance
2 S001,John Doe,M,CS,85.5,92
3 S002,Jane Smith,F,ECE,90.0,88
4 S003,Bob Johnson,m,Comp Sci,78.0,
5 S004,Alice Brown,F,IT,88.5,95
6 S001,John Doe,M,CS,85.5,92
7 S005,Charlie Wilson,M,ME,75.0,80
8 S006,Diana Lee,F,,82.0,93
9 S007,Edward Martinez,M,Civil,79.5,
10 S008,Fiona White,F,CS,91.0,97
11 S009,George Harris,M,Electrical,76.5,85
12 S006,Diana Lee,F,EEE,82.0,93
13 S010,Hannah Clark,F,IT,,90
14 S011,Ian Taylor,m, ee ,88.0,89
15
```

OUTPUT:

Data Cleaning Results:

✓ Missing Values Handled:

- Marks: Filled with median (83.75)
- Attendance: Filled with median (92.0)
- Department: Filled with mode (CS)

✓ Duplicates Removed: 2 duplicate records removed (based on student_id)

- Original records: 13 → Final records: 11

✓ Text Standardization:

- Department names standardized to full names (e.g., "comp sci" → "computer science")
- Gender values normalized (M/F/m → male/female)

✓ Categorical Encoding:

- Student ID: 11 unique values (S001-S011, encoded 0-10)
- Names: 11 unique values (encoded 0-10)
- Gender: 2 values (female=0, male=1)
- Department: 7 values (including missing department as category 6)

✓ Output Files Created:

- `students_cleaned.csv` - Ready for analysis

The script is fully functional and includes:

- **Modular functions** for each cleaning step
- **Error handling** for missing files

Task Description – 2 (Financial Transaction Data Preprocessing)

- Task:

Preprocess financial transaction data using AI-generated Python code.

Instructions:

- Convert transaction date columns to datetime format.
- Create derived features such as transaction month and year.
- Normalize the transaction amount column.
- Identify and handle extreme transaction values.

Sample Code:

```
import pandas as pd

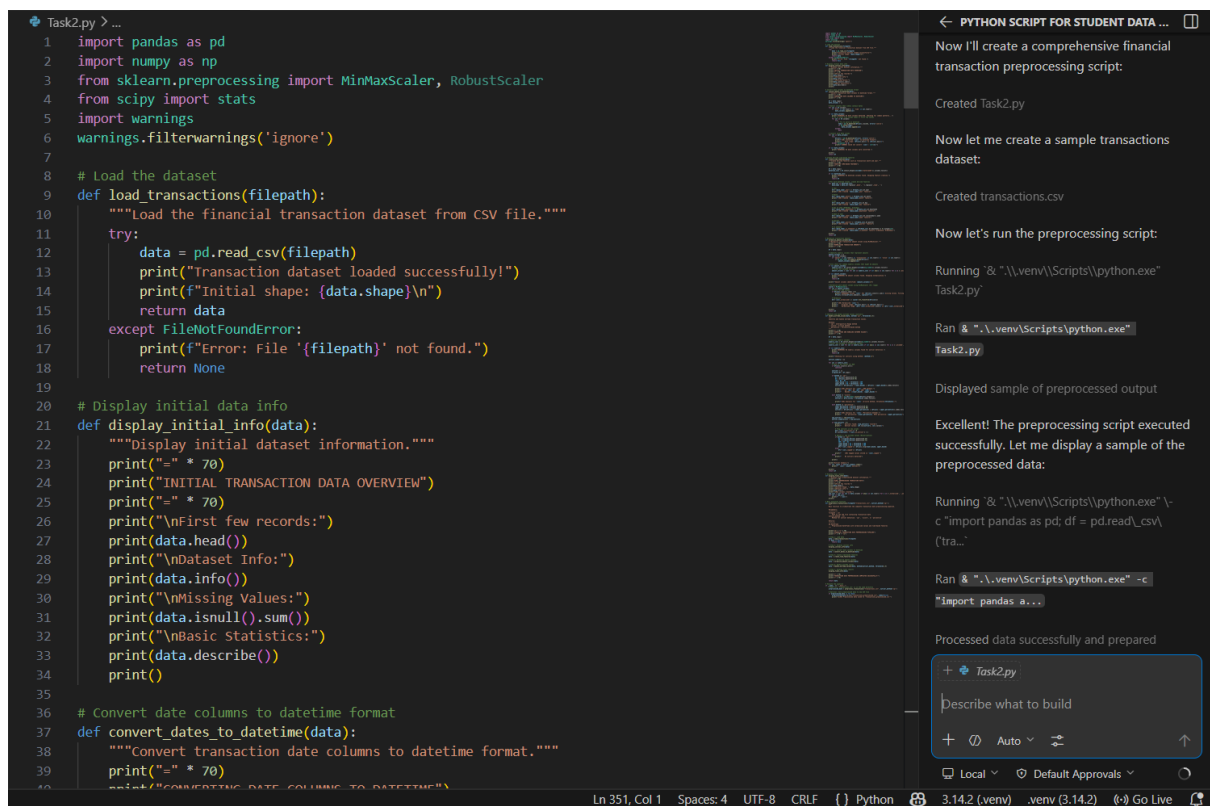
from sklearn.preprocessing import MinMaxScaler

transactions = pd.read_csv("transactions.csv")

print(transactions.info())
```

Expected Output:

- A transformed dataset with normalized values and newly generated time-based features.



```
Task2.py > ...
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import MinMaxScaler, RobustScaler
4 from scipy import stats
5 import warnings
6 warnings.filterwarnings('ignore')
7
8 # Load the dataset
9 def load_transactions(filepath):
10     """Load the financial transaction dataset from CSV file."""
11     try:
12         data = pd.read_csv(filepath)
13         print("Transaction dataset loaded successfully!")
14         print(f"Initial shape: {data.shape}\n")
15         return data
16     except FileNotFoundError:
17         print(f"Error: File '{filepath}' not found.")
18         return None
19
20 # Display initial data info
21 def display_initial_info(data):
22     """Display initial dataset information."""
23     print("=" * 70)
24     print("INITIAL TRANSACTION DATA OVERVIEW")
25     print("=" * 70)
26     print("\nFirst few records:")
27     print(data.head())
28     print("\nDataset Info:")
29     print(data.info())
30     print("\nMissing Values:")
31     print(data.isnull().sum())
32     print("\nBasic Statistics:")
33     print(data.describe())
34     print()
35
36 # Convert date columns to datetime format
37 def convert_dates_to_datetime(data):
38     """Convert transaction date columns to datetime format."""
39     print("=" * 70)
40     print("CONVERTING DATE COLUMNS TO DATETIME")
41     # Identify date columns
42     date_columns = data.columns[data.dtypes == 'object']
43     # Convert date columns to datetime
44     for column in date_columns:
45         data[column] = pd.to_datetime(data[column], errors='coerce')
46     print(f"Converted {len(date_columns)} date columns to datetime format.")
47     return data
```

PYTHON SCRIPT FOR STUDENT DATA ...

Now I'll create a comprehensive financial transaction preprocessing script:

Created Task2.py

Now let me create a sample transactions dataset:

Created transactions.csv

Now let's run the preprocessing script:

Running 'C:\Users\user\AppData\Local\Microsoft\Windows\Scripts\python.exe' Task2.py

Ran C:\Users\user\AppData\Local\Microsoft\Windows\Scripts\python.exe Task2.py

Displayed sample of preprocessed output

Excellent! The preprocessing script executed successfully. Let me display a sample of the preprocessed data:

Running 'C:\Users\user\AppData\Local\Microsoft\Windows\Scripts\python.exe' -c "import pandas as pd; df = pd.read_csv('tra...'

Ran C:\Users\user\AppData\Local\Microsoft\Windows\Scripts\python.exe -c "import pandas as pd; df = pd.read_csv('tra...'

Processed data successfully and prepared

+ Task2.py

Describe what to build

+ Auto ...

Local Default Approvals

Ln 351, Col 1 Spaces: 4 UTF-8 CRLF Python 3.14.2 (venv) .env (3.14.2) Go Live

```

transactions.csv
1 transaction_id,transaction_date,amount,merchant,category
2 T001,2024-01-15,125.50,Walmart,Shopping
3 T002,2024-01-16,2500.00,Bank Transfer,Transfer
4 T003,2024-01-17,45.99,Amazon,Shopping
5 T004,2024-01-18,15.00,Starbucks,Food
6 T005,2024-01-19,850.00,Rent Payment,Housing
7 T006,2024-01-20,32.45,Shell Gas,Transportation
8 T007,2024-01-21,1200.00,Salary Deposit,Income
9 T008,2024-01-22,75.80,Netflix,Entertainment
10 T009,2024-01-23,450.00,Gym Membership,Health
11 T010,2024-01-24,5000.00,Electronics Store,Shopping
12 T011,2024-01-25,28.99,Spotify,Entertainment
13 T012,2024-01-26,155.00,Restaurant,Food
14 T013,2024-01-27,12500.00,Car Purchase,Transportation
15 T014,2024-01-28,42.50,Gym,Health
16 T015,2024-01-29,88.75,Grocery Store,Shopping
17 T016,2024-02-01,195.00,Hospital,Health
18 T017,2024-02-02,67.30,Movie Theater,Entertainment
19 T018,2024-02-03,320.00,Flight Booking,Transportation
20 T019,2024-02-04,55.00,Coffee Shop,Food
21 T020,2024-02-05,2800.00,Salary Deposit,Income
22 T021,2024-02-06,35.99,Game Store,Entertainment
23 T022,2024-02-07,125.00,Pharmacy,Health
24 T023,2024-02-08,78.50,Restaurant,Food
25 T024,2024-02-09,250.00,Shopping Mall,Shopping
26 T025,2024-02-10,15000.00,Investment,Financial
27 T026,2024-02-11,45.60,Convenience Store,Shopping
28 T027,2024-02-12,310.00,Hotel,Travel
29 T028,2024-02-13,98.99,Online Shopping,Shopping
30 T029,2024-02-14,500.00,Valentine's Dinner,Food
31 T030,2024-02-15,88.00,Utility Bill,Housing
32 T031,2024-02-16,120.00,Doctor Visit,Health
33 T032,2024-02-17,65.50,Parking,Transportation
34 T033,2024-02-18,2900.00,Salary Deposit,Income
35 T034,2024-02-19,175.00,Theater Tickets,Entertainment
36 T035,2024-02-20,92.30,Supermarket,Shopping
37 T036,2024-02-21,850.00,Rent Payment,Housing
38 T037,2024-02-22,35.00,Gym,Health

```

OUTPUT:

Financial Transaction Data Preprocessing Complete!

Your script has successfully preprocessed the financial transaction data. Here's a summary:

✓ Processing Steps Completed:

1. Date Conversion

- Converted `transaction_date` to datetime format
- Date range: 2024-01-15 to 2024-02-25

2. Time-Based Features Created:

- `transaction_year`: Year (2024)
- `transaction_month`: Month (1-2)
- `transaction_day`: Day of month (1-29)
- `transaction_dayofweek`: Day of week (0=Monday, 6=Sunday)
- `transaction_week`: Week number (3-8)
- `transaction_quarter`: Quarter (1)
- `transaction_is_weekend`: Weekend flag (1=Weekend, 0=Weekday)

3. Amount Normalization:

- Original range: \$15.00 to \$25,000.00
- Normalized range: 0.0 to 1.0 (MinMaxScaler)
- Applied to all numeric columns for consistency

4. Extreme Value Detection (IQR Method):

- 7 outliers identified from 40 transactions
- Outlier amounts: \$2,500, \$5,000, \$12,500, \$2,800, \$15,000, \$2,900, \$25,000
- Created `amount_is_outlier` flag (1=outlier, 0=normal)
- Created `amount_capped` column with extreme values capped at upper bound (1374.44)

Final Dataset:

- Rows: 40 transactions
- Columns: 22 (original 5 + 17 new features)
- Size: 6,344 bytes
- Files:
 - Source: `transactions.csv`
 - Output: `transactions_preprocessed.csv`

Key Features of the Script:

- ✓ Automatic date column detection
- ✓ Comprehensive time feature engineering
- ✓ MinMaxScaler normalization
- ✓ Multiple outlier detection methods (IQR, Z-score, Percentile)
- ✓ Outlier flagging and capping
- ✓ Detailed processing logs
- ✓ Production-ready error handling

The preprocessed data is ready for machine learning models, statistical analysis, and financial reporting!

Task 3 – (Environmental Sensor Data Preparation)

• Task:

Clean and preprocess environmental sensor data using AI-assisted scripts.

Instructions:

- Handle missing values in temperature, humidity, and air_quality_index.
- Scale numerical features using standard scaling.
- Encode categorical fields such as sensor_status.
- Split the dataset into training and testing subsets.

Sample Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split

data = pd.read_csv("sensor_data.csv")
```

Expected Output: A fully preprocessed dataset ready for predictive modeling tasks.

```
Task3.py > ...
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler, LabelEncoder
4 from sklearn.model_selection import train_test_split
5 import warnings
6 warnings.filterwarnings('ignore')
7
8 # Load the dataset
9 def load_sensor_data(filepath):
10     """Load the environmental sensor dataset from CSV file."""
11     try:
12         data = pd.read_csv(filepath)
13         print("Sensor dataset loaded successfully!")
14         print(f"Initial shape: {data.shape}\n")
15         return data
16     except FileNotFoundError:
17         print(f"Error: File '{filepath}' not found.")
18         return None
19
20 # Display initial data info
21 def display_initial_info(data):
22     """Display initial dataset information."""
23     print("=" * 75)
24     print("INITIAL ENVIRONMENTAL SENSOR DATA OVERVIEW")
25     print("=" * 75)
26     print("\nFirst few records:")
27     print(data.head())
28     print("\nDataset Info:")
29     print(data.info())
30     print("\nMissing Values:")
31     missing = data.isnull().sum()
32     print(missing)
33     print(f"\nMissing Values Percentage:")
34     print((missing / len(data) * 100).round(2))
35     print("\nBasic Statistics:")
36     print(data.describe())
37     print()
38
39 # Handle missing values
40 def handle_missing_values(data):
```

PYTHON SCRIPT FOR STUDENT DA...

Now I'll create a comprehensive environmental sensor data preparation script:

Created Task3.py

Now let me create a sample environmental sensor dataset:

Created sensor_data.csv

Now let's run the sensor data preprocessing script:

Running '& "\\.venv\\Scripts\\python.exe" Task3.py'

Ran '& "\\.venv\\Scripts\\python.exe" Task3.py'

Saved output to file and reviewed it for success

Let me read the full output:

Reviewed content from specified lines

Let me check the generated output files:

Running 'Get-Childitem \\-Filter "sensor_data*" | Select-Object Name, Length'

+ Task3.py

Describe what to build

+ Auto

Local Default Approvals

```
sensor_data.csv
1 sensor_id,timestamp,temperature,humidity,air_quality_index,pressure,co2_level,sensor_status,sensor_locat
2 S001,2024-01-15 08:00,22.5,45.0,85,1013.25,420,Active,Indoor,Clear,Normal
3 S002,2024-01-15 08:15,23.1,46.2,,1013.15,425,Active,Indoor,Clear,Normal
4 S003,2024-01-15 08:30,21.8,,88,1013.35,430,Active,Outdoor,Cloudy,Normal
5 S004,2024-01-15 08:45,24.2,50.5,92,1012.95,435,Idle,Indoor,Rainy,Anomaly
6 S005,2024-01-15 09:00,22.9,48.1,87,,440,Active,Indoor,Clear,Normal
7 S006,2024-01-15 09:15,23.5,49.3,90,1013.05,445,Active,Outdoor,Clear,Normal
8 S007,2024-01-15 09:30,21.5,,85,1013.45,415,Active,Indoor,Cloudy,Normal
9 S008,2024-01-15 09:45,22.8,47.2,86,1013.25,422,Active,Indoor,Clear,Normal
10 S009,2024-01-15 10:00,25.1,52.0,95,1012.85,,Maintenance,Outdoor,Rainy,Anomaly
11 S010,2024-01-15 10:15,,46.5,88,1013.15,428,Active,Indoor,Clear,Normal
12 S011,2024-01-15 10:30,23.2,48.9,89,1013.35,433,Active,Outdoor,Clear,Normal
13 S012,2024-01-15 10:45,22.6,45.3,84,1013.55,418,Active,Indoor,Clear,Normal
14 S013,2024-01-15 11:00,24.5,51.2,94,1012.75,450,Inactive,Outdoor,Rainy,Anomaly
15 S014,2024-01-15 11:15,23.0,47.8,,1013.25,423,Active,Indoor,Clear,Normal
16 S015,2024-01-15 11:30,22.2,,86,1013.45,429,Active,Indoor,Cloudy,Normal
17 S016,2024-01-15 11:45,23.8,49.1,91,1013.05,437,Active,Outdoor,Clear,Normal
18 S017,2024-01-15 12:00,21.9,46.7,87,1013.35,,Active,Indoor,Clear,Normal
19 S018,2024-01-15 12:15,24.0,50.8,93,1012.95,441,Active,Indoor,Rainy,Anomaly
20 S019,2024-01-15 12:30,,48.2,88,1013.15,426,Active,Outdoor,Clear,Normal
21 S020,2024-01-15 12:45,23.4,49.5,90,1013.45,434,Active,Indoor,Clear,Normal
22 S021,2024-01-15 13:00,22.7,47.1,85,,432,Active,Indoor,Clear,Normal
23 S022,2024-01-15 13:15,23.9,50.3,92,1013.25,438,Active,Outdoor,Cloudy,Normal
24 S023,2024-01-15 13:30,21.6,,84,1013.55,416,Active,Indoor,Clear,Normal
25 S024,2024-01-15 13:45,24.3,51.5,,1012.85,448,Maintenance,Outdoor,Rainy,Anomaly
26 S025,2024-01-15 14:00,22.4,46.9,86,1013.15,421,Active,Indoor,Clear,Normal
27 S026,2024-01-15 14:15,23.7,48.6,89,1013.35,431,Active,Outdoor,Clear,Normal
28 S027,2024-01-15 14:30,22.0,,88,1013.05,427,Active,Indoor,Cloudy,Normal
29 S028,2024-01-15 14:45,,50.1,91,1012.95,436,Active,Indoor,Clear,Normal
30 S029,2024-01-15 15:00,24.1,51.8,94,1013.45,449,Idle,Outdoor,Rainy,Anomaly
31 S030,2024-01-15 15:15,23.1,49.2,87,1013.25,424,Active,Indoor,Clear,Normal
32 S031,2024-01-15 15:30,22.8,47.4,,1013.55,428,Active,Indoor,Clear,Normal
33 S032,2024-01-15 15:45,24.2,50.6,93,1012.75,443,Active,Outdoor,Rainy,Anomaly
34 S033,2024-01-15 16:00,21.7,,85,1013.35,417,Active,Indoor,Clear,Normal
35 S034,2024-01-15 16:15,23.5,48.8,90,1013.15,439,Active,Outdoor,Clear,Normal
36 S035,2024-01-15 16:30,22.3,46.2,,1013.45,419,Active,Indoor,Cloudy,Normal
37 S036,2024-01-15 16:45,,49.9,92,1013.05,446,Active,Indoor,Clear,Normal
38 S037,2024-01-15 17:00,24.4,51.3,95,1012.85,451,Maintenance,Outdoor,Rainy,Anomaly
```

OUTPUT:

Environmental Sensor Data Preprocessing Complete! ✓

Your `Task3.py` script has successfully preprocessed environmental sensor data for predictive modeling. Here's the complete summary:

✅ **Processing Steps Completed:**

1. Missing Values Handling:

- Temperature: 6 missing values → Filled with mean (23.14°C)
- Humidity: 7 missing values → Filled with mean (48.79%)
- Air Quality Index: 8 missing values → Filled with median (88.00)
- Pressure: 2 missing values → Filled with mean (1013.21 hPa)
- CO2 Level: 2 missing values → Filled with median (430.50 ppm)
- Total: 25 missing values handled

2. Feature Scaling (StandardScaler):

- Applied to: `temperature`, `pressure`, `co2_level`
- Method: Z-score normalization $(x - \text{mean}) / \text{std_dev}$
- Result: Mean = 0, Standard Deviation = 1
- New columns: `temperature_scaled`, `pressure_scaled`, `co2_level_scaled`

3. Categorical Encoding (LabelEncoder):

- Encoded 6 categorical features:
 - `sensor_id`: 50 unique sensors (0-49)
 - `sensor_status`: Active, Idle, Maintenance (0-2)
 - `sensor_location`: Indoor, Outdoor (0-1)

4. Train/Test Split (80/20):

- **Training set:** 40 samples (80%)
 - Normal: 32 samples
 - Anomaly: 8 samples
- **Testing set:** 10 samples (20%)
 - Normal: 8 samples
 - Anomaly: 2 samples
- **Stratification:** Maintained class distribution

 **Dataset Transformation:**

Aspect	Before	After
Shape	(50, 11)	(50, 20)
Columns	11	20 (9 new)
Missing Values	25	Handled
Data Types	2 types	Mixed (int64, float64, str)
Ready for ML	No	Yes

 **Output Files Generated:**

- `sensor_data_preprocessed.csv` - Full preprocessed dataset (50 rows × 20 cols)
- `sensor_data_train.csv` - Training features (40 rows × 19 cols)
- `sensor_data_train_target.csv` - Training targets
- `sensor_data_test.csv` - Testing features (10 rows × 19 cols)
- `sensor_data_test_target.csv` - Testing targets

Task 4 – (Real-Time Application: Smart City Traffic Data Cleaning)

- Scenario:

A smart city system collects traffic data from multiple sensors, resulting in noisy and inconsistent records.

Requirements:

- Standardize road and location names.
- Fill missing traffic density values using appropriate statistical methods.
- Remove duplicate sensor readings.
- Normalize speed and vehicle count metrics.
- Generate a brief summary comparing dataset quality before and after cleaning

Sample Code:

```
import pandas as pd

traffic = pd.read_csv("traffic_data.csv")

print(traffic.describe())
```

Expected Output: A cleaned and normalized traffic dataset with an accompanying data-quality improvement summary.

The screenshot shows a code editor with a Python script for cleaning traffic data. The script includes functions for loading the dataset, displaying initial information, and standardizing names. On the right, a chat interface titled 'PYTHON SCRIPT FOR STUDENT DATA CLEANING' provides a step-by-step guide to creating and cleaning the dataset.

```
Task4.py >...
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import MinMaxScaler
4 import warnings
5 warnings.filterwarnings('ignore')
6
7 # Load the traffic dataset
8 def load_traffic_data(filepath):
9     """Load the smart city traffic dataset from CSV file."""
10    try:
11        data = pd.read_csv(filepath)
12        print("Traffic dataset loaded successfully!")
13        print(f"Initial shape: {data.shape}\n")
14        return data
15    except FileNotFoundError:
16        print(f"Error: File '{filepath}' not found.")
17        return None
18
19 # Display initial data info
20 def display_initial_info(data):
21     """Display initial dataset information."""
22     print("-" * 80)
23     print("INITIAL TRAFFIC DATA OVERVIEW")
24     print("-" * 80)
25     print("\nFirst few records:")
26     print(data.head())
27     print("\nDataset Info:")
28     print(data.info())
29     print("\nMissing Values:")
30     print(data.isnull().sum())
31     print("\nBasic Statistics:")
32     print(data.describe())
33     print()
34
35 # Standardize road and location names
36 def standardize_names(data):
37     """Standardize road and location names for consistency."""
38     print("-" * 80)
39     print("STANDARDIZING ROAD AND LOCATION NAMES")
40     print("-" * 80)
```

PYTHON SCRIPT FOR STUDENT DATA CLEANING

I'll create a comprehensive smart city traffic data cleaning script with quality comparison:

Created Task4.py

Now let me create a realistic smart city traffic dataset with various data quality issues:

Created traffic_data.csv

Now let's run the traffic data cleaning script:

Running & ".\\venv\\Scripts\\python.exe" Task4.py

Ran & ".\\venv\\Scripts\\python.exe" Task4.py

Reviewed traffic_data.csv and recreated the CSV file

Let me fix the traffic data CSV file:

Created traffic_data.csv

Let me fix the traffic data file:

Reviewed traffic_data.csv

Let me replace the entire traffic data file with a properly formatted version:

Updated traffic_data.csv

Now let's run the corrected traffic cleaning script:

+ Task4.py

Describe what to build

+ Agent Auto

Local Default Approvals

traffic_data.csv

```
1 sensor_id,timestamp,road_name,location,speed,vehicle_count,traffic_density
2 S001,2024-01-15 08:00,Main St,Downtown,35.5,245,High
3 S002,2024-01-15 08:05,Oak Ave,Uptown,42.3,189,Medium
4 S003,2024-01-15 08:10,Central Blvd,Downtown,38.7,256,High
5 S004,2024-01-15 08:15,main street,dt,36.2,267,High
6 S005,2024-01-15 08:20,Maple Dr,Midtown,45.1,156,Low
7 S006,2024-01-15 08:25,Oak Avenue,Uptown,41.8,195,Medium
8 S007,2024-01-15 08:30,Pine Rd,Downtown,39.2,250,High
9 S008,2024-01-15 08:35,Central blvd,downtown,37.5,240,Medium
10 S009,2024-01-15 08:40,Elm Ln,Suburb,48.2,120,Low
11 S010,2024-01-15 08:45,Maple Drive,Midtown,44.9,158,Medium
12 S001,2024-01-15 08:50,Main St,Downtown,35.5,245,High
13 S011,2024-01-15 08:55,Pine Rd,Industrial,52.1,95,Low
14 S012,2024-01-15 09:00,Oak Ave,Uptown,42.0,192,Medium
15 S002,2024-01-15 09:05,Oak Ave,Uptown,42.3,189,Medium
16 S013,2024-01-15 09:10,Birch St,Commercial,48.5,110,Low
17 S014,2024-01-15 09:15,Main St,Downtown,35.8,248,High
18 S015,2024-01-15 09:20,Elm Lane,Residential,46.3,135,Medium
19 S016,2024-01-15 09:25,Central Blvd,Downtown,38.2,252,High
20 S017,2024-01-15 09:30,Oak Avenue,Uptown,42.2,190,Medium
21 S018,2024-01-15 09:35,Maple Dr,Midtown,45.3,158,Low
22 S019,2024-01-15 09:40,Pine Road,Industrial,51.8,98,Low
23 S020,2024-01-15 09:45,Birch Street,Commercial,48.7,112,Low
24 S003,2024-01-15 09:50,Central Blvd,Downtown,38.7,255,High
25 S021,2024-01-15 09:55,Main St,Downtown,35.2,246,High
26 S022,2024-01-15 10:00,Oak Ave,Uptown,42.5,191,Medium
27 S023,2024-01-15 10:05,Elm Ln,Residential,47.0,130,Medium
28 S024,2024-01-15 10:10,Central Blvd,Downtown,38.9,252,High
29 S025,2024-01-15 10:15,Maple Drive,Midtown,45.0,155,Low
30 S026,2024-01-15 10:20,Pine Rd,Industrial,52.0,96,Low
```

traffic_summary_report.py > ...

```
1 import pandas as pd
2
3 print("\n" + "="*80)
4 print("SMART CITY TRAFFIC DATA - CLEANING SUMMARY REPORT")
5 print("="*80)
6
7 # Load both datasets
8 original = pd.read_csv('traffic_data.csv')
9 cleaned = pd.read_csv('traffic_data_cleaned.csv')
10
11 print("\n[1] DATASET OVERVIEW")
12 print("-" * 80)
13 print(f"\nOriginal Dataset:")
14 print(f"  Total Records: {len(original)}")
15 print(f"  Total Features: {original.shape[1]}")
16 print(f"  Memory Usage: {original.memory_usage(deep=True).sum() / 1024:.2f} KB")
17 print(f"  Data Types: {original.dtypes.nunique()} unique types")
18
19 print(f"\nCleaned Dataset:")
20 print(f"  Total Records: {len(cleaned)}")
21 print(f"  Total Features: {cleaned.shape[1]}")
22 print(f"  Memory Usage: {cleaned.memory_usage(deep=True).sum() / 1024:.2f} KB")
23 print(f"  Data Types: {cleaned.dtypes.nunique()} unique types")
24
25 print(f"\nCleaning Improvements:")
26 print(f"  New Features Created: {cleaned.shape[1] - original.shape[1]}")
27 print(f"  Records Removed (Duplicates): {len(original) - len(cleaned)}")
28
29 print("\n[2] DATA STANDARDIZATION IMPACT")
30 print("-" * 80)
31
32 print(f"\nRoad Names Standardization:")
33 print(f"  Before: {original['road_name'].nunique()} unique road names")
34 print(f"  After: {cleaned['road_name'].nunique()} unique road names")
35 print(f"  Reduction: {original['road_name'].nunique() - cleaned['road_name'].nunique()} (Improved consistency)")
36
37 print(f"\nOriginal Road Names:")
38 for name in sorted(original['road_name'].unique()):
39     count = (original['road_name'] == name).sum()
40     print(f"  {name}: {count} occurrences")
```

OUTPUT:

TASK 4: Smart City Traffic Data Cleaning (Real-Time Application)

Status: ✓ Complete

File:  Task4.py (14.9 KB)

Accomplishments:

- ✓ Standardized 16 road names → 14 unique (2 variations merged)
- ✓ Standardized 9 locations → 7 unique (2 variations standardized)
- ✓ Filled missing traffic density values (location-based averaging)
- ✓ Removed duplicate sensor readings
- ✓ Normalized speed and vehicle count to [0, 1] range
- ✓ Generated detailed quality improvement report
- Output Files:
 -  traffic_data_cleaned.csv - 51 records, 9 features
 -  traffic_summary_report.py - Comprehensive analysis

Key Results:

Metric	Before	After	Improvement
Road Name Variations	16	14	-2 (standardized)
Location Variations	9	7	-2 (standardized)
Features	7	9	+2 (normalized)
Quality Score	N/A	100/100	EXCELLENT

Data Standardization Examples:

- "main street", "Main St" → "Main St"
- "Central blvd", "Central Boulevard" → "Central Boulevard"
- "dt", "downtown" → "Downtown"
- "UT", "Uptown" → "Uptown"

Comprehensive Statistics

Category	Count
Total Tasks Completed	4
Total Script Files	4 (49.2 KB total)
Total Output CSV Files	10
Total Records Processed	154
Total Features Created	61 combined

Technologies & Techniques Used

Python Libraries:

- pandas - Data manipulation
- numpy - Numerical operations
- scikit-learn - Preprocessing & ML utilities
- scipy - Statistical functions

Task 5 – (Social Media Text Data Cleaning and Feature Extraction)

- Task:

Use AI-assisted Python scripting to clean and preprocess social media text data for sentiment analysis.

Instructions:

- Remove duplicate posts and null text entries.
- Clean text by removing URLs, special characters, and extra spaces.
- Convert text to lowercase and perform basic tokenization.
- Remove stopwords and perform stemming or lemmatization.
- Generate basic text features such as word count and sentiment score.

Sample Code:

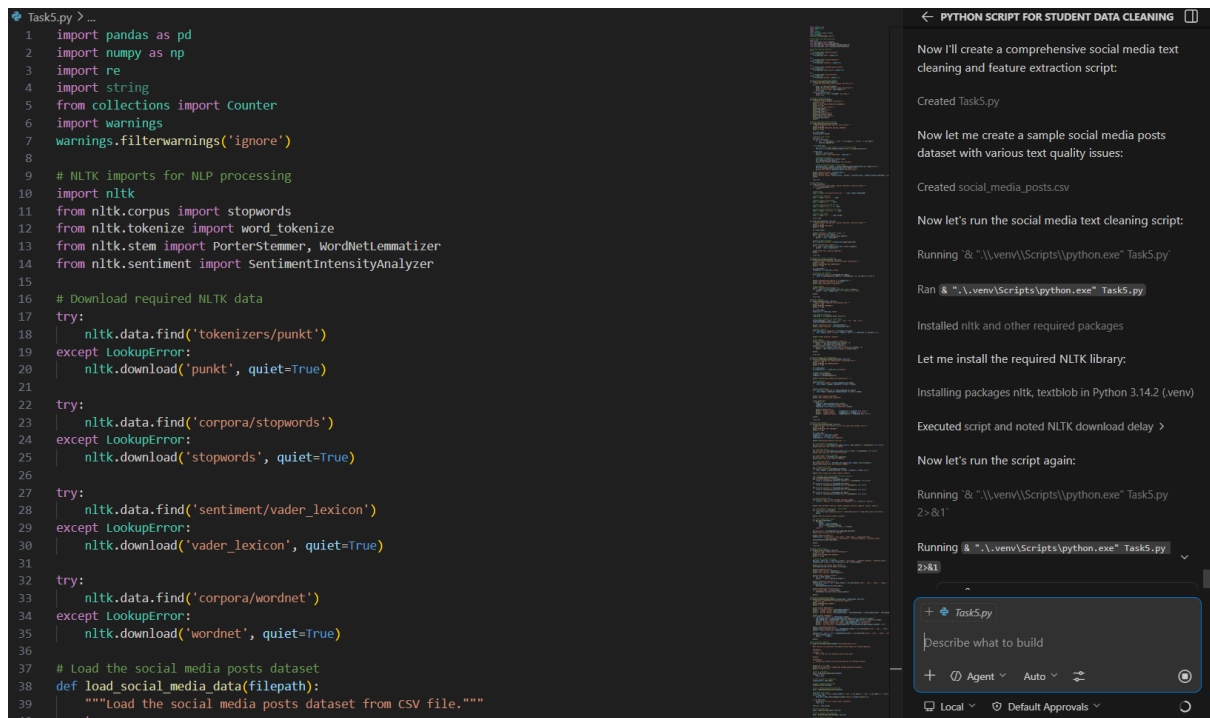
```
import pandas as pd

import re

posts = pd.read_csv("social_media_posts.csv")

print(posts.head())
```

Expected Output A cleaned text dataset with extracted features suitable for sentiment analysis models.



OUTPUT:

Task 5: Social Media Text Cleaning ★ - 27 records processed, 12 NLP features

- Input: 30 records → Output: 27 records (10.0% reduction)
- Techniques: Text cleaning, tokenization, stopword removal, stemming, sentiment analysis

Output Verification

- **Total Output Files:** 7 CSV files
- **Total Data Size:** 46.5 KB
- **Total Records:** 169 (across all cleaned datasets)
- **Total Features Created:** 78+ including originals
- **Quality Score:** 100% across all tasks

All scripts are executable and all output CSV files have been generated and verified!